

Activity 09: ACL and secure management

TGS-2023037854 | v7.0 | 13 September 2026 | 110 minutes

Goal

Evaluate ordered ACL rules and secure VTY access, including AAA client/server roles and safe fallback.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, S1-baseline.cfg, S2-baseline.cfg, R1-baseline.cfg, R2-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 2 x Router2911 (R1,R2), 2 x Switch2960 (S1,S2), 2 x PC-PT (PC1,PC2). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
S1 FastEthernet0/24	R1 GigabitEthernet0/0	Straight-through
R1 GigabitEthernet0/1	R2 GigabitEthernet0/1	Cross-over/Auto
R2 GigabitEthernet0/0	S2 FastEthernet0/24	Straight-through
S2 FastEthernet0/1	PC2 FastEthernet0	Straight-through

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
PC1	10.10.10.10	255.255.255.0	10.10.10.1	none
PC2	10.10.20.20	255.255.255.0	10.10.20.1	none

For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste the following device-specific blocks into each named device CLI one device at a time. Wait for the router/switch prompt between blocks. These are complete activity configurations for the stated construction. Do not apply configuration from a different activity. When copy asks for Destination filename, press Enter. Save the Packet Tracer working file after all blocks.

Device S1

```
enable
configure terminal
hostname S1
no ip domain-lookup
interface range FastEthernet0/1, FastEthernet0/24
  switchport mode access
  no shutdown
exit
end
copy running-config startup-config
```

Device S2

```
enable
```

```
configure terminal
hostname S2
no ip domain-lookup
interface range FastEthernet0/1, FastEthernet0/24
  switchport mode access
  no shutdown
exit
end
copy running-config startup-config
```

Device R1

```
enable
configure terminal
hostname R1
no ip domain-lookup
interface GigabitEthernet0/0
  ip address 10.10.10.1 255.255.255.0
  no shutdown
exit
interface GigabitEthernet0/1
  ip address 10.0.0.1 255.255.255.252
  no shutdown
exit
ip route 10.10.20.0 255.255.255.0 10.0.0.2
ip domain-name example.test
username learner privilege 1 secret CcnaPracticeOnly2026!
enable secret CcnaEnableOnly2026!
crypto key generate rsa modulus 1024
ip ssh version 2
ip access-list standard MGMT-SOURCES
  permit 10.10.10.10
  deny any
exit
line vty 0 4
  login local
  transport input ssh
  access-class MGMT-SOURCES in
  exec-timeout 5 0
exit
end
copy running-config startup-config
```

Device R2

```
enable
configure terminal
hostname R2
no ip domain-lookup
interface GigabitEthernet0/0
  ip address 10.10.20.1 255.255.255.0
  no shutdown
exit
interface GigabitEthernet0/1
  ip address 10.0.0.2 255.255.255.252
  no shutdown
exit
ip route 10.10.10.0 255.255.255.0 10.0.0.1
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config
```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

PC1 Command Prompt: ssh -l learner 10.10.10.1 (include a space between username and address) and enter the isolated training password from the R1 block; expect User EXEC prompt and a privilege-limited session. PC2 ssh -l learner 10.10.10.1 must fail because the VTY ACL permits only 10.10.10.10. PC1 telnet 10.10.10.1 must fail. Both directions ping remain permitted. R1 show access-lists MGMT-SOURCES shows permitted/denied counters; show ip ssh shows version 2. If PT rejects inline modulus, use crypto key generate rsa and answer 1024 to the prompt. Production exercise uses 2048+ where supported.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

On R1 remove the VTY access-class temporarily: configure terminal; line vty 0 4; no access-class MGMT-SOURCES in; end. PC2 SSH to 10.10.10.1 now reaches authentication; it previously should be blocked. This is a controlled policy failure. Restore before any other changes. Then demonstrate first-match ACL order in the offline flow matrix.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

Restore access-class MGMT-SOURCES in under R1 line vty 0 4. Keep transport input ssh; do not permit Telnet. Use the direct CLI/console for recovery if a remote session is denied.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. how ordered filtering, SSH transport, local authentication and command privilege protect different boundaries

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain how ordered filtering, SSH transport, local authentication and command privilege protect different boundaries. Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

9. Complete the separately labelled extension

The fixed credentials in device blocks are deliberately public isolated sandbox examples; replace them locally and never reuse them. AAA client/server role and RADIUS1812 are transition fixture practice. For real AAA use a trainer-provided server baseline and test console/local recovery before selecting a remote method list. An authentication reject is not the same as server unavailability.

Record supported platform, output observed or evidence-only limitation.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open activity09-acl-secure-management; inspect topology.md, case.json and observations.csv. Identify the controls for: Evaluate ordered ACL rules and secure VTY access, including AAA client/server roles and safe fallback.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

Replace broad permit with the narrow SSH permit shown in reference.json, then explicit deny ip. Set transport [ssh] and timeout_minutes 5. Preserve console access before any AAA experiment. Explain why local fallback usually applies to unavailable server, not rejected credentials.

Open case.json in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0. Domain-specific computed fields appear in JSON detail.

Capture evidence.txt, corrected case.json and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying faulty.json over case.json. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

flow_verdicts: ["permit", "deny", "deny"]

AAA_client: "router"

AAA_server: "192.0.2.181"

Required PASS controls

- ACL first-match flow matrix
- SSH only and bounded session
- AAA client and server roles
- tested local fallback

Transition practice

Refresh extension: the router/switch is an AAA client requesting service from a RADIUS/TACACS+ server. This fixture models roles and fallback; it does not connect to an authentication service.

Configuration files

The complete per-device configurations are shown in the construction steps and included as named *-baseline.cfg files. specimen.cfg combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python ipaddress documentation. Exact URLs and source links are in README.md.